This is a demo Final Project document!

Text in blue indicates an Instruction. (Don’t include this in the actual document!)
Text in red indicates a note by TA. (Also don’t include this too)
The other is the part you must fill in!.

Group Member

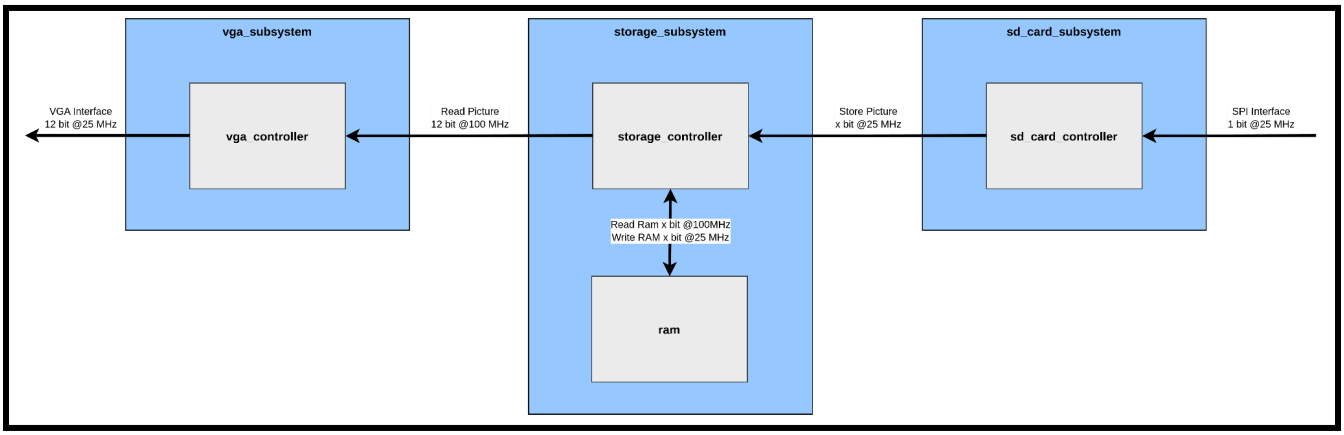
Group name : <Insert your group name here>

Name	Student Number

Overall Design Block Diagram

Provide the overall design block diagram, which serves as an abstraction of your design—each module is represented as a block. Then, describe how these modules interact with each other. Additionally, specify the data transfer frequency.

An example is provided below.



Original picture :
https://drive.google.com/file/d/1N1F1RvsEUcBtE0W0M58z3MniMEtaaghy/view?usp=sharing
<Provide the link in case picture is too small.>
<The block diagram provided above is for reference only and has not been tested.>

Design Decision.

In this section, explain the reasoning behind your design choices. Consider the following aspects when justifying your implementation:

1. Module Architecture
    - Why did you structure your design this way?
    - What are the advantages of using this approach? (e.g., modularity, efficiency, ease of debugging)
2. Communication Protocols
    - Why did you choose this specific protocol (e.g., SPI, I2C, UART,AXI) for communication?
    - How does this protocol suit your design requirements?
3. Clock and Data Transfer Rate
    - Why did you select this specific clock frequency?
    - How does the data rate impact system performance?
    - Did you consider constraints such as FPGA timing, external device compatibility, or noise margins?
4. Resource Utilization
    - How does your design balance resource usage (LUTs, FFs, BRAMs) and performance?
    - Why did you optimize certain parts of the design?

An example 1 design decision is provided below.

Design Decision : Using native Simple dual port RAM
Reason :

We chose a **Simple Dual-Port RAM** because it provides two interfaces:

- **Interface A** → Supports both **read and write** operations.
- **Interface B** → Supports **read-only** operations.

Why This Fits Our Design

- The **VGA controller** only needs to **read** data.
- The **SPI controller** only needs to **write** data.
- Since each module operates on a separate port, there are no access conflicts, making **Simple Dual-Port RAM** an ideal choice.

Why Native Interface Over AXI?

We used the **native interface** instead of AXI because it has **lower complexity** and requires fewer resources, making implementation easier while still meeting our performance requirements.

<You must write every design decision that you take>

<We recommend that students carefully read the specifications and documentation before starting their design.>

Implementation Detail

In this section, Provide the code and description for each module that is in your design.
Example below.
Module : single_pulser
Module code :

```
module single_pulser (
   input wire data_in, // input data
   input wire clk,     // clock
   input wire rst,     // active low reset
   output wire data_out // output data
);

 reg prev_data = 0;
 reg data_out_reg = 0;

 assign data_out = data_out_reg;

 always @(posedge clk) begin
   if (rst) begin
     data_out_reg <= 0;
     prev_data <= 0;
   end else begin
     prev_data <= data_in;
     if (data_in == 1 && prev_data == 0) begin
       data_out_reg <= 1;
     end else begin
       data_out_reg <= 0;
     end
   end
 end

endmodule
```

Module Description :
Generate a pulse of 1 clock cycle for the input data.

Challenge Faced

In this section, Provide what challenges have you faced during this project.

1. There is a lot of documentation to read.
2. The Basys3 board just broke down.
3. TA is too harsh.

<The above is just for example only>